

Cross domain content extraction with fake captcha

Cross domain content extraction with fake captcha - Author not stated, blog.kotowicz.net.

- Published: date not stated
- Original: <<http://blog.kotowicz.net/2011/07/cross-domain-content-extraction-with.html>>
- Preserved from: <http://blog.kotowicz.net/2011/07/cross-domain-content-extraction-with.html> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

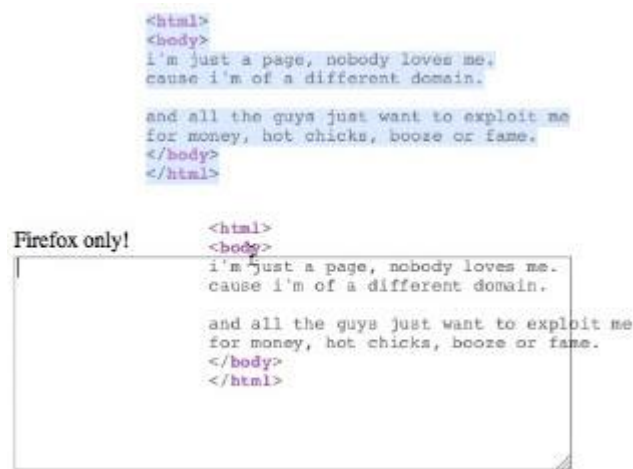
UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

[Content extraction](#) is one of the recently documented UI redressing vectors. It exploits Firefox vulnerability that allows to display any URL **HTML source** in an iframe like this:

```
<iframe src="view-source:http://any-page-you.like/cookies-included">
```

With social engineering attacker tricks user into selecting (usually invisible) page source and dragging it to attackers' controlled textarea. A simple demo is [here](#):

|



| | | Drag & drop other page source (cross-domain) | |

Once attacker gets the page source dropped into his textarea, he may begin to extract contents (like session IDs, user names, anti csrf tokens etc.) and launch further attacks.

However, this way of using the vector requires significant effort from a user and **is pretty difficult to exploit** in real world situation (there's some clicking and dragging involved). Also, it will stop working once Mozilla [disallows cross origin drag & dropping](#).

I've found a neat way to do cross-origin content extraction that might be more suitable for some classes of websites. Ladies and gentleman, let me present **Fake Captcha**:

No more drag

The weak point of the 'classic' method for me was the dragging that was involved. In Firefox, once you drag something, it displays a shadow of the object at the cursor - and a whole HTML source being displayed for the user is really hard to hide. I decided to convince the user to **copy & paste** the source with his clipboard instead.

Copying & pasting requires four steps:

- selecting the text to copy
- ctrl-c
- navigating to target element
- ctrl-v

Each of these steps requires user intervention. I could make a game/quiz that requires certain keypresses, but that's weak (although it [works for Facebook users](#)). Instead, I wanted it to feel natural for the user. Nothing is hidden and he just **uses the clipboard because he wants to**.

So, when do you use a clipboard?

Well, I don't like typing. So everytime I'm forced to repeat my e-mail address in a form, I just copypaste it. I decided to go that way. What if we display longish captcha-like 'security code' for a user to retype? 16 characters or more? Some of them will skip this step altogether, some will retype, but **most will select the text and copy/paste**.

[[image: image]]

(https://blogger.googleusercontent.com/img/b/R29vZ2xl/AVvXsEjJk3XPcqZooGwNj7Lvrl-yydaD2YLjNmIAJEyOcj-kKmzYt705XwzS1QKw_xISFcOqswvFO_GLEIyeopR63nIWnMnTkIMR3HDUY7scGkjtFCXX6pZxs3J6DnV LewaWg2V6wsC7csWabfQ/s1600/fake-captcha1.png)

How do you select?

You can select with your mouse. In Firefox, you can also select by double / tripple clicking. My assumption is that most of the users use the clicking method to select text.

Double click stops at word boundary, third click expands to whole paragraph (try this text). In the above example, you need three clicks to select the whole visible code. Why do we care?

I'm framed!

Because the security code input field is just precisely positioned part of the view-source:d victim page. And by tripple clicking user selects the whole line from the page source!



Demo

It's best to see [the demo](#) to understand what's going on. We want to extract the anti-CSRF token from the [victim page](#) cross domain. The token is in the page source, line 7:

```
<!DOCTYPE html PUBLIC "-//W3C//DTD HTML 4.01 Transitional//EN" "http://www.w3.org/TR/html4/loose.dtd">
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<title>NDCP</title>
<script type="text/javascript">
var csrf_token = '35fb6df6-2ab9-408b-abe3-769412a58e15';
</script>
<style>
body {
  background: url(nuke.jpg) left top repeat;
  color: white;
  font-family: Verdana, arial, sans-serif;
}
// and so on
```

So we display the source in a small frame, position it to only display a few characters, starting from line 7, column 19. Then we convince the user to select the whole line with tripple click - double click will stop at minus sign, so the user will probably do the third click to select all. After selecting he copies, clicks the next field and pastes. Then we're done.

Details matter

See [the source](#) to appreciate all the **small, but very important details**, especially:

- how to measure the font size used in view-source:
- what was view-source:view-source: used for
- how to position an iframe to line / column of HTML source
- how the input and frame was styled to look similar

How not to get owned?

- web developers - use [X-Frame-Options](#) header (js framebusting won't work here). Remember, once you allow your site to be framed, you're opening to a whole class of UI redressing attacks, most of the attacks are not even discovered yet, it's a new field of research. So **if you don't use X-Frame-Options, better have a really good explanation.**
- users - don't use Firefox or look carefully on what you do use [NoScript](#)

Summary

There's a new 'fake captcha' method of using the [content extraction](#) UI redressing vector.

Pros:

- does not require drag & dropping
- accounts for font-size differences
- more convincing for a user

Cons:

- won't work if user uses mouse to select text (unless attacker is interested in only the visible part)
- requires a captcha like string in victim HTML source
- it's line / column position must be constant and known to attacker
- only one line of HTML source might be copied (but websites' HTML is often minimized to a single line)

You might find the requirements very limiting. I also thought that's simply impossible to exploit in real life. **Until I started looking** - wait for the [next post](#) :)

**Update:* **Latest NoScript (2.1.2+)* contains code [neutralizing fake captcha method](#). Yeat another great work of [Giorgio Maone](#)!

***Update 2:* ***Fake CAPTCHA technique spotted in the wild to [extract Facebook CSRF tokens](#).*